

INTEGRAÇÃO DO COCOTB À IDE AURORA: COSSIMULAÇÃO E VERIFICAÇÃO DE MODELOS DE HARDWARE EM PYTHON

Arthur A. Martins[1] - arthur.araujo@estudante.ufjf.br
Chrysthofer A. A. Afonso[1] - chrysthofer.afonso@estudante.ufjf.br
Victor M. Ribeiro[1] - victor.mendes@ufjf.br
Nelson C. N. de Assis[1] - nelson.assis@estudante.ufjf.br
Guilherme B. Morett[2] - guilherme.morett@iprj.uerj.br
Luciano M. de Andrade Filho[1] - luciano.andrade@ufjf.br
[1]Núcleo de Instrumentação e Processamento de Sinais (NIPS-CERN),
Universidade Federal de Juiz de Fora – Juiz de Fora – MG
[2]Instituto Politécnico, Universidade do Estado do Rio de Janeiro – Nova Friburgo – RJ

Resumo. *Verificar um algoritmo embarcado em hardware é, em essência, um problema de modelagem computacional: o mesmo estimador existe como modelo estrutural sintetizável, executado por um simulador de eventos discretos, e como modelo numérico de referência, e a confiança no circuito nasce do confronto contínuo entre os dois. Este trabalho apresenta a integração do cocotb (coroutine cosimulation testbench) à IDE AURORA, do ecossistema do processador soft-core SAPHO, que permite escrever esse confronto inteiramente em Python, em cossimulação com os simuladores Icarus Verilog e Verilator, com a rotina de teste reconhecida pela interface e executada em um clique. A metodologia é demonstrada na verificação de uma Unidade de Medição Fasorial implementada no SAPHO segundo a norma IEC/IEEE 60255-118-1: gerador de estímulos, modelo de referência em dupla precisão, emulação da aquisição e análise convivem em um único testbench, que valida o hardware em dois níveis, contra o fasor analítico da norma e contra o modelo numérico. O primeiro nível confirmou um Erro Total do Vetor máximo de 0,98%, dentro do limite normativo; o segundo mostra que o float de 24 bits do processador reproduz a referência com desvio de 5×10^{-6} no fasor, atribuindo o erro remanescente ao método de estimação, e não à aritmética do hardware.*

Keywords: *Modelagem e verificação de sistemas digitais, Cossimulação, cocotb, Processador soft-core SAPHO, Unidade de Medição Fasorial.*

1. INTRODUÇÃO

No projeto de *hardware* digital, a simulação é o momento em que um modelo é posto à prova. O circuito, expresso em uma linguagem de descrição de *hardware* (*Hardware Description Language*, HDL) como Verilog ou VHDL, constitui esse modelo, estrutural e em nível de transferência de registradores (*Register-Transfer Level*, RTL); a rotina de teste (*testbench*) é o aparato experimental que o envolve, aplicando estímulos e confrontando as respostas com o comportamento esperado, antes de qualquer implementação física.

Tradicionalmente, esse aparato é descrito na mesma HDL do circuito. A curva de aprendizado é acentuada, e construir *testbenches* sofisticados em Verilog ou VHDL costuma ser trabalhoso, sobretudo quando o ensaio exige gerar sinais complexos ou analisar estatisticamente os resultados; isso afasta parte dos projetistas do potencial da simulação.

Por outro lado, linguagens de alto nível como Python dispõem de um ecossistema maduro de bibliotecas para geração de sinais e análise de dados, como NumPy e SciPy (Harris et al., 2020; Virtanen et al., 2020), que seriam de grande valia se acopladas aos estímulos e à análise dos ensaios de *hardware*. É dessa convergência que surge o cocotb (*coroutine cosimulation testbench*), um arcabouço que permite escrever *testbenches* inteiramente em Python, em cossimulação (simulação conjunta) com simuladores HDL convencionais (cocotb contributors, 2024). Em vez de descrever o *testbench* em Verilog, o projetista manipula os sinais do circuito sob teste (*Design Under Test*, DUT) por meio de corrotinas Python (funções assíncronas, que pausam e retomam a execução), aproveitando todo o poder de processamento e de análise de dados da linguagem. Trata-se de um recurso amplamente validado pela comunidade de verificação de *hardware* e adotado, inclusive, em projetos de física de altas energias.

O Núcleo de Instrumentação e Processamento de Sinais (NIPS-CERN)[1], da Universidade Federal de Juiz de Fora (UFJF), desenvolve um conjunto próprio de ferramentas de código aberto para a síntese e a simulação de circuitos complexos em Verilog, organizado em torno do processador *soft-core* SAPHO (*Scalable-Architecture Processor for Hardware Optimization*), um núcleo embarcado descrito em HDL e sintetizado sob demanda (Kapisch et al., 2026), já validado em aplicações como a geração de fractais em FPGA (Andrade Filho et al., 2025) e a simulação de pulsos do TileCal em tempo real (Ribeiro and Andrade Filho, 2025). Esse fluxo é orquestrado pela AURORA (*Advanced Utility Running Optimized Resource Architectures*), a IDE (*Integrated Development Environment*) construída em Electron (*framework* para aplicações de desktop), que integra a cadeia compiladora YANC (*Yet Another Compiler*) e os simuladores Icarus Verilog e Verilator, além de visualizadores de formas de onda como o GTKWave e o Surfer e do PRISM, um visualizador de RTL desenvolvido internamente (Andrade Filho et al., 2025). Todo esse instrumental é distribuído de forma portátil por meio de um ambiente MSYS2/MinGW64 (de tipo Unix, com compilador C/C++, para o Windows) customizado pelo laboratório, que reúne, em um único pacote, a cadeia de compilação, os simuladores e um interpretador Python dedicado.

Para tornar esse ambiente ainda mais poderoso, este trabalho propõe a integração, de forma transparente e *user-friendly*, das simulações em cocotb a todo o ferramental da AURORA, incluindo a sua assistente de inteligência artificial, a Aurora Intelligence. Como o cocotb foi concebido para ambientes Unix, a sua adoção reaproveita esse mesmo pacote portátil, o que viabiliza a execução nativa no Windows. Com essa integração, o usuário passa a escrever o seu próprio *testbench* em Python, ou a solicitar que a assistente o gere, aproveitando as bibliotecas desse ecossistema diretamente sobre os circuitos do SAPHO, sem abrir mão do fluxo de compilação e de visualização já consolidado na ferramenta.

O restante do artigo está organizado da seguinte forma: a Seção 2 apresenta o cocotb, com um exemplo introdutório; a Seção 3 descreve a IDE AURORA e a sua inteligência artificial; a Seção 4 detalha a integração do cocotb à AURORA; a Seção 5 expõe a metodologia de modelagem e verificação aplicada a uma PMU; a Seção 6 apresenta os resultados; a Seção 7

[1] Em parceria com o CERN (Organização Europeia para a Pesquisa Nuclear, em francês *Conseil Européen pour la Recherche Nucléaire*). Mais informações em `https://www.nipscern.com/projects/aurora`.

aponta os trabalhos futuros; e a Seção 8 reúne as conclusões.

2. O COCOTB

No cocotb, cada ensaio é uma corrotina que dialoga com o simulador: aplica estímulos ao DUT, aguarda o avanço do tempo de simulação e confere as respostas, enquanto o simulador subjacente executa o modelo do circuito (cocotb contributors, 2024).

Na prática, uma função Python decorada com `@cocotb.test()` torna-se um caso de teste. Por meio de um manipulador do DUT, o código acessa as portas do circuito, atribuindo e lendo valores em `.value`, e utiliza gatilhos (*triggers*), como o `Timer`, para suspender a corrotina até um instante de simulação. A Listagem 1 ilustra um *testbench* mínimo para um somador: aplica dois operandos, aguarda dez nanossegundos e confere o resultado.

Listagem 1: Testbench mínimo em cocotb para um somador.

```python
import cocotb
from cocotb.triggers import Timer

@cocotb.test()
async def test_soma(dut):
    dut.a.value = 3
    dut.b.value = 5
    await Timer(10, units="ns")
    assert dut.soma.value == 8
```

Nesse exemplo, `dut` representa o módulo Verilog sob teste, e tanto os estímulos quanto a verificação são expressos diretamente em Python, sem nenhuma linha adicional de HDL.

Além do avanço de tempo com o `Timer`, o cocotb oferece gatilhos ligados ao próprio circuito, como `RisingEdge` e `FallingEdge`, que suspendem a corrotina até uma borda do sinal de relógio (*clock*), e o utilitário `Clock`, que gera um relógio de período fixo. Corrotinas independentes podem ser postas a executar em paralelo com `cocotb.start_soon`, o que permite modelar, por exemplo, um gerador de estímulos e um monitor de saídas operando ao mesmo tempo. A Listagem 2 ilustra um teste com relógio e reinicialização (*reset*) síncrona de um contador.

Listagem 2: Testbench cocotb de um contador, com relógio e *reset*.

```python
import cocotb
from cocotb.clock import Clock
from cocotb.triggers import FallingEdge

@cocotb.test()
async def test_contador(dut):
    Clock(dut.clk, 10, units="ns").start()
    dut.rst.value = 1
    await FallingEdge(dut.clk) # aplica o reset
    dut.rst.value = 0
    for _ in range(5):
        await FallingEdge(dut.clk)
    assert dut.q.value == 5
```

Em ambos os exemplos, a lógica de teste, com os seus laços, condições e verificações, é escrita em Python comum, enquanto o simulador HDL apenas executa o modelo do circuito a cada avanço de tempo. É essa separação que permite acoplar à verificação bibliotecas como a NumPy, para gerar sinais e conferir resultados numéricos, tarefa trabalhosa de reproduzir em um *testbench* descrito em HDL.

3. A IDE AURORA

O cocotb foi integrado à AURORA, a IDE de código aberto que organiza o fluxo de projeto do ecossistema SAPHO. Construída em Electron, a AURORA reúne, sob uma única interface, a edição de código, a cadeia compiladora YANC, os simuladores Icarus Verilog e Verilator e os visualizadores de formas de onda, poupando o projetista de operar manualmente cada ferramenta da linha de comando (Andrade Filho et al., 2025).

A IDE integra ainda a Aurora Intelligence, assistente de inteligência artificial especializada no fluxo do SAPHO. Além de auxiliar na escrita e na depuração do código, a assistente pode gerar *testbenches* em cocotb a pedido do usuário, aproximando a verificação baseada em Python do restante do ambiente. É sobre essa base que se constrói a integração descrita na próxima seção.

4. INTEGRAÇÃO DO COCOTB À AURORA

A integração parte da forma como o cocotb opera: em vez de compilar um *testbench* em Verilog, ele carrega o *testbench* em Python no próprio simulador HDL por meio da interface procedural do Verilog (*Verilog Procedural Interface*, VPI), controlando o DUT durante a simulação. A AURORA oferece esse fluxo sobre os seus dois motores: o Icarus Verilog, por um módulo VPI dinâmico, e o Verilator, que exigiu a construção de um módulo VPI próprio para o Windows, cujo detalhamento foge ao escopo deste trabalho e será tratado em publicação específica.

Do ponto de vista do usuário, a integração é transparente. A árvore de arquivos do projeto passou a reconhecer arquivos Python: ao detectar as marcas de um *testbench* cocotb, isto é, a importação do módulo e o decorador `@cocotb.test`, a IDE habilita a simulação diretamente, sem configuração manual. O circuito a ser simulado é indicado por uma diretiva em comentário no próprio arquivo (`# aurora-toplevel:`) ou herdado da configuração do projeto.

Internamente, ao acionar a simulação, a AURORA gera em tempo de execução um roteiro Python que invoca o executor do cocotb, seleciona o simulador desejado e repassa os dados do projeto por variáveis de ambiente. Todo o processo usa o interpretador e os simuladores já embutidos na ferramenta. Como resultado, o projetista seleciona o arquivo `.py` e executa a simulação com um clique, obtendo, quando desejado, as formas de onda no formato FST (cerca de dez vezes mais compacto que o VCD tradicional) para inspeção no GTKWave ou no Surfer. A Figura 1 mostra a IDE com um *testbench* cocotb selecionado, pronto para ser simulado.

5. METODOLOGIA: MODELAGEM E VERIFICAÇÃO DE UMA PMU

Para demonstrar o fluxo em um caso realista, o cocotb foi empregado na verificação de uma Unidade de Medição Fasorial (*Phasor Measurement Unit*, PMU) implementada no SAPHO. Uma PMU estima, a partir de um sinal elétrico amostrado, o seu fasor (magnitude e fase), a frequência e a taxa de variação da frequência (*Rate of Change of Frequency*, ROCOF), grandezas empregadas no monitoramento de sistemas de potência em tempo real (Phadke and Thorp, 2008) segundo a norma IEC/IEEE 60255-118-1 (IEC/IEEE, 2018). O algoritmo, originado de um trabalho em MATLAB (Fonseca, 2025), foi portado para a linguagem C± do SAPHO com o apoio da Aurora Intelligence, e o projeto completo está disponível publicamente[2]. Vale registrar que esse porte exercita o próprio ambiente descrito neste trabalho:

[2] `https://github.com/Chrysthofer/proj_PMU_padrao`.

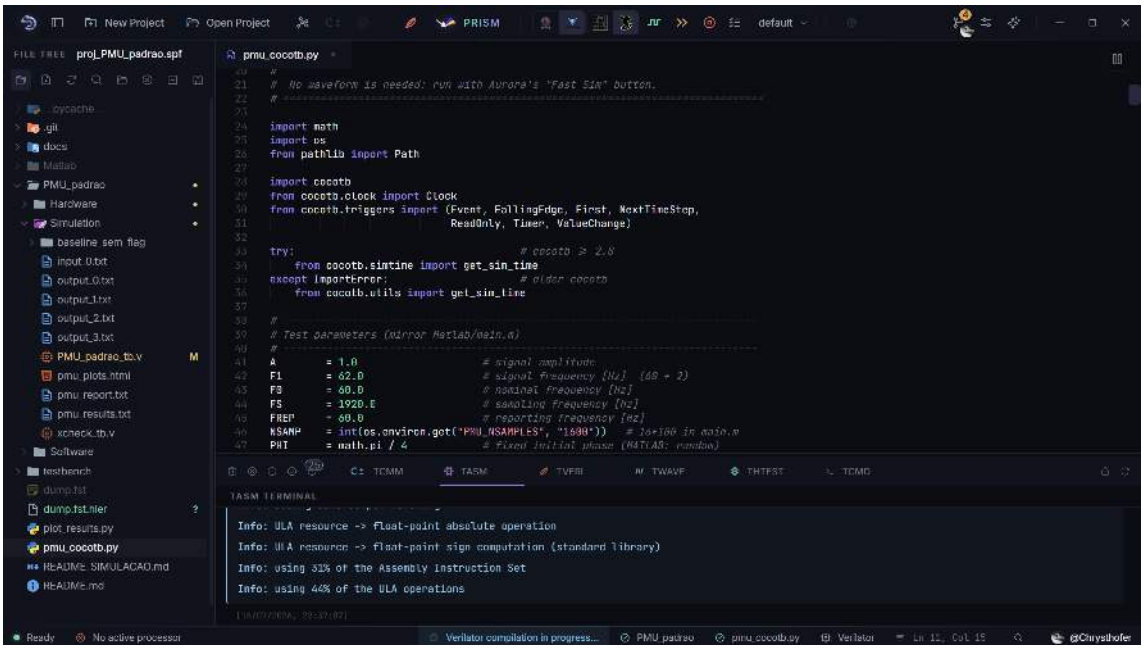


Figura 1: A IDE AURORA com um *testbench* cocotb (arquivo `.py`) reconhecido na árvore de arquivos, pronto para ser simulado sem sair da interface. No editor aparecem os parâmetros do ensaio da norma usados neste trabalho ($F_1 = 62\,\text{Hz}$, $F_s = 1920\,\text{Hz}$, 1.600 amostras) e, na barra inferior, o Verilator em compilação.

por conhecer a arquitetura do SAPHO, a cadeia de compiladores YANC e a API da AURORA, a assistente opera com conhecimento de domínio que um assistente genérico não teria, e a tradução de um algoritmo de referência para a linguagem do processador deixa de ser uma barreira de entrada.

O algoritmo segue a abordagem clássica de estimação por demodulação. O sinal amostrado é multiplicado por senos e cossenos na frequência nominal (60 Hz), o que desloca a componente de interesse para a banda base, e o resultado passa por um filtro de resposta finita ao impulso (*Finite Impulse Response*, FIR) da classe M da norma (Roscoe et al., 2013), que suaviza o sinal e rejeita as componentes indesejadas. A frequência é obtida da derivada da fase do fasor entre amostras consecutivas, e a ROCOF, da variação da frequência. A norma IEC/IEEE 60255-118-1 estabelece os limites de conformidade por três métricas: o Erro Total do Vetor (*Total Vector Error*, TVE), que combina em um único número os erros de magnitude e de fase; o Erro de Frequência (*Frequency Error*, FE); e o Erro da Taxa de Variação da Frequência (*Rate of Change of Frequency Error*, RFE). Este trabalho avalia as duas primeiras, em regime permanente; a avaliação do RFE e a bateria de ensaios dinâmicos previstos na norma ficam como desdobramento, conforme a Seção 7. O ensaio emprega o sinal de teste da norma amostrado a 1.920 amostras por segundo, ou seja, 32 amostras por ciclo na frequência nominal. Do ponto de vista do processador, trata-se de uma carga de trabalho representativa de processamento digital de sinais, com um laço de filtragem intensivo em multiplicações e somas de ponto flutuante, executado a cada amostra em tempo real.

O que torna esse caso ilustrativo é que toda a verificação foi escrita em Python, sobre o processador embarcado. A Figura 2 detalha a arquitetura do *testbench* e a Listagem 3 a resume em código, nos mesmos quatro passos: o estímulo da norma, o modelo de referência, o acionamento do processador e a verificação dos limites. De todo o conjunto, apenas um bloco é *hardware*.

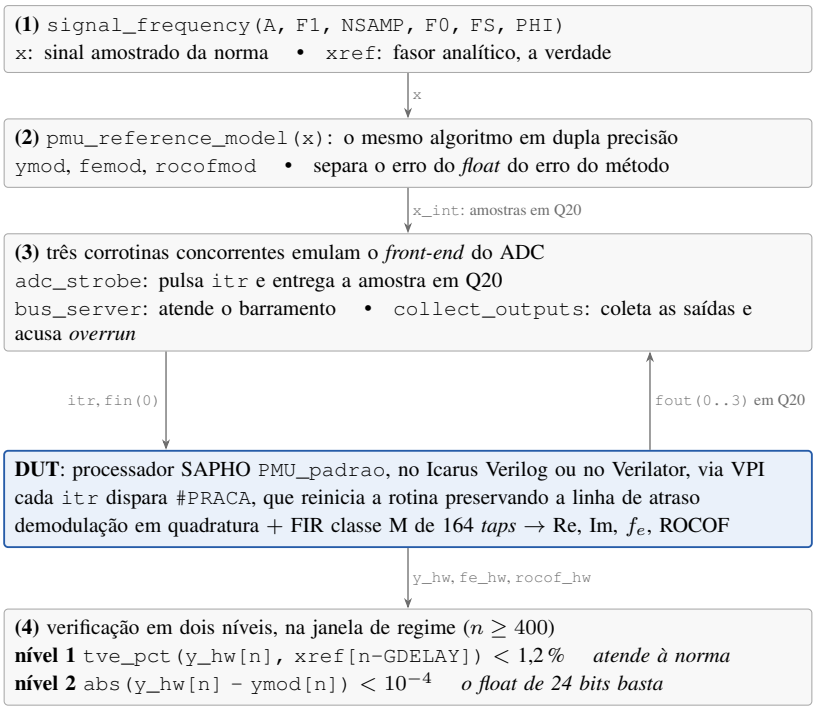


Figura 2: Arquitetura do *testbench* cocotb da PMU. Os passos (1) a (4) correspondem aos da Listagem 3. Todos os blocos são Python, exceto o DUT (em azul), o único *hardware* do conjunto: à esquerda desce o estímulo, a interrupção e a amostra em Q20, e à direita sobem as quatro saídas do processador. O passo (4) fecha nos dois níveis, contra a verdade analítica (`xref`) e contra o modelo em dupla precisão (`ymod`).

Fonte: Autores (2026)

Listagem 3: *Testbench* cocotb da PMU (resumido). Os passos (1) a (4) são os da Figura 2.

```python
@cocotb.test()
async def pmu_signal_frequency(dut):
    # (1) sinal da norma e fasor de referencia analitico
    x, xref = signal_frequency(A, F1, NSAMP, F0, FS, PHI)
    # (2) o mesmo algoritmo em dupla precisao
    ymod, femod, rocofmod = pmu_reference_model(x)
    # (3) tres corrotinas emulam o ADC e acionam o SAPHO
    cocotb.start_soon(adc_strobe(dut, x_int, adc, STROBE_CLKS))
    cocotb.start_soon(bus_server(dut, adc))
    cocotb.start_soon(collect_outputs(dut, outs, adc, done, NSAMP))
    await First(done.wait(), Timer(timeout_ns, "ns"))
    assert adc["overruns"] == 0
    # (4) verificacao em dois niveis, na janela de regime
    tve = {n: tve_pct(y_hw[n], xref[n-GDELAY]) for n in win}
    dre = max(abs(y_hw[n].real - ymod[n].real) for n in win)
```

Vale destacar a emulação, em Python, do circuito de aquisição que, na FPGA, entregaria as amostras ao processador: as três corrotinas do passo (3) reproduzem o protocolo de sincronismo do conversor analógico-digital com meia dúzia de linhas. Se o processador não consumir uma amostra a tempo, a emulação acusa a perda (*overrun*), o que permite verificar, ainda em simulação, se o *clock* escolhido dá conta da taxa de amostragem. Reproduzir esse protocolo em um *testbench* descrito em HDL seria bem mais verboso.

A verificação é feita em dois níveis, o que só é prático porque o modelo de referência vive no mesmo ambiente Python. No primeiro, a saída do processador é comparada ao fasor de referência analítico pelo TVE; esse nível responde pela *conformidade* da implementação. No segundo, a saída do *hardware* é comparada ao modelo de dupla precisão do mesmo algoritmo; esse nível responde pela *fidelidade numérica*, separando o erro da aritmética de 24 bits do

SAPHO do erro intrínseco ao método de estimação. A distinção importa porque um resultado fora do limite normativo pode ter duas origens muito diferentes, e apenas o segundo nível permite atribuí-la.

6. RESULTADOS E DISCUSSÃO

A Tabela 1 reúne os resultados na janela de regime permanente.

Tabela 1: Verificação da PMU no SAPHO via cocotb, na janela de regime permanente.

Métrica	Valor	Limite
TVE máximo (vs. referência analítica)	0,98 %	1 %
Viés de frequência (FE)	$<$0,001 Hz	0,005 Hz
Fasor: *float* 24 bits vs. dupla precisão	5×10^{-6}	10^{-4}
Frequência: *float* 24 bits vs. dupla precisão	7×10^{-4} Hz	10^{-2}
Ciclos por amostra	$\approx$4.600	–
Clock mínimo para tempo real	8,8 MHz	–

Fonte: Autores (2026)

Os números confirmam a implementação: o TVE máximo de 0,98 % permanece dentro do limite normativo.

O segundo nível de comparação produz um resultado de interesse próprio para quem for reimplementar o algoritmo. O *float* de 24 bits do SAPHO reproduz o modelo de dupla precisão com diferença máxima da ordem de 5×10^{-6} no fasor e de 7×10^{-4} Hz na frequência, mais de três ordens de grandeza abaixo do erro observado contra a referência analítica. Disso decorrem duas conclusões. A primeira é que o formato numérico reduzido do processador não é o fator limitante: a precisão de 24 bits basta para esta classe de algoritmo, e ampliá-la não melhoraria a conformidade. A segunda é que o erro remanescente no TVE tem origem no *método* de estimação por demodulação, e não na aritmética do *hardware*. Sem o modelo de referência convivendo no mesmo ambiente, essa atribuição não seria possível, e o projetista poderia gastar esforço otimizando a representação numérica sem ganho algum.

Quanto ao custo computacional, cada amostra é processada em cerca de 4.600 ciclos, de modo que um *clock* de apenas 8,8 MHz basta para operar na taxa de 1.920 amostras por segundo, folgado para qualquer FPGA moderna.

A inspeção das formas de onda, disponível no mesmo fluxo, permite acompanhar visualmente o comportamento do processador durante a simulação. A Figura 3 mostra os sinais da PMU no visualizador Surfer: o sinal de entrada e as saídas de fasor, frequência e ROCOF acompanham a variação de frequência do ensaio sem descontinuidades aparentes.

O que a inspeção visual sugere, a comparação com o modelo de referência quantifica. Por conviver no mesmo ambiente Python e partir do mesmo estímulo da norma, o modelo em dupla precisão fornece, a cada amostra, o valor que o algoritmo produziria sem as limitações da aritmética do processador, e o desvio em relação à saída do SAPHO é calculado diretamente com as rotinas numéricas da própria linguagem (Virtanen et al., 2020; Hunter, 2007). A Figura 4 reúne esse confronto nos dois níveis ao longo do ensaio. A concordância mantém-se em toda a janela de regime permanente, sem acúmulo progressivo de erro, o que confirma no domínio do tempo o que a Tabela 1 resume em valores máximos.

Cabe uma observação honesta sobre o desempenho: o cocotb não acelera a simulação, pois recorre aos mesmos Icarus Verilog ou Verilator por baixo, e em circuitos grandes o seu custo é maior que o de um *testbench* nativo. O ganho está na acessibilidade e na produtividade. Toda a cadeia de verificação, do gerador de sinais ao modelo de referência e à análise estatística, coube em algumas centenas de linhas de Python legível, sem uma única linha de *testbench* em HDL e

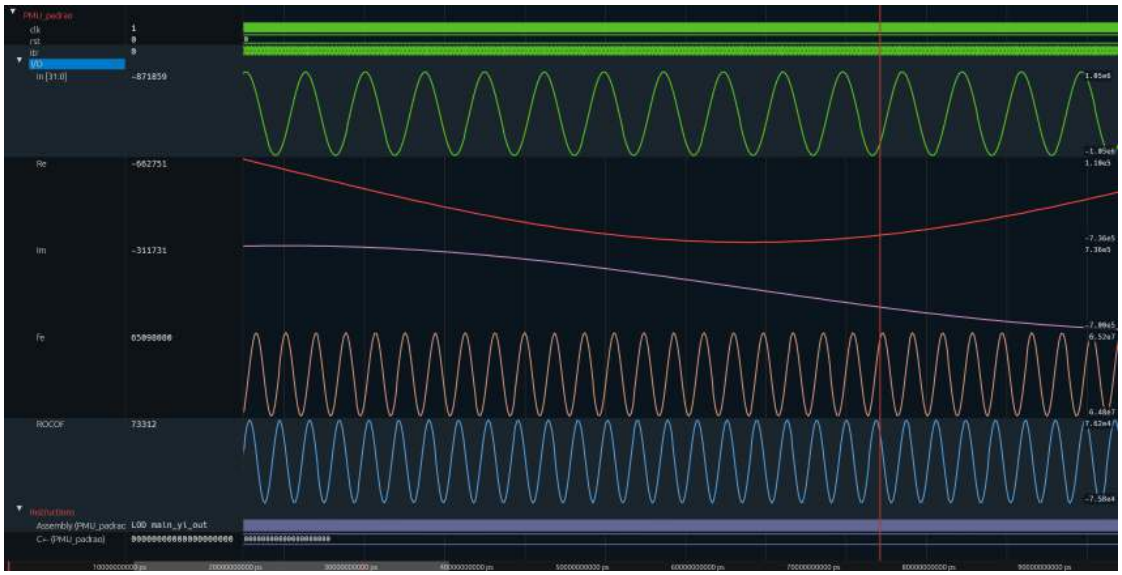


Figura 3: Formas de onda da simulação da PMU inspecionadas no Surfer: o sinal de entrada e as saídas do processador (fasor, frequência e ROCOF).

aproveitando bibliotecas maduras da linguagem. Para um projeto embarcado com requisitos de norma, isso reduz de forma expressiva a barreira de entrada da verificação.

Vista sob a ótica da modelagem computacional, a metodologia coloca dois modelos do mesmo estimador em execução conjunta: o modelo em nível de registrador, sintetizável, descrito para o SAPHO, e um modelo numérico de referência em dupla precisão, escrito na mesma linguagem do *testbench*. A cossimulação confronta os dois de forma contínua, e a separação do erro em uma parcela de método e outra de aritmética decorre justamente dessa convivência. Arcabouços que descrevem o próprio *hardware* em Python, como o PyMTL3 (Jiang et al., 2020), exploram caminho afim ao aproximar a modelagem da implementação; aqui a implementação permanece em HDL sintetizável e ao Python cabe a modelagem de referência e a análise, papel para o qual o seu ecossistema numérico se mostra particularmente adequado.

7. TRABALHOS FUTUROS

A integração abre várias frentes. O módulo VPI construído para executar o Verilator com o cocotb no Windows é, por si só, um resultado de engenharia que merece descrição em publicação própria. No plano da verificação, a geração automática de *testbenches* pela Aurora Intelligence, a partir de uma descrição em linguagem natural, pode reduzir ainda mais a barreira de entrada. Por fim, o caso da PMU pode ser estendido à bateria completa de ensaios estáticos e dinâmicos da norma e à síntese em FPGA, para quantificar o consumo de recursos lógicos e a frequência máxima de operação.

8. CONCLUSÕES

Este trabalho integrou o cocotb à IDE AURORA e, com isso, trouxe para um mesmo ambiente os dois lados da verificação de um modelo embarcado: o modelo estrutural, executado pelo simulador, e o modelo de referência que o julga. Os *testbenches* para os circuitos do SAPHO passam a ser escritos inteiramente em Python e executados de forma transparente sobre os simuladores Icarus Verilog e Verilator, sem que o usuário opere a linha de comando. A metodologia foi demonstrada na verificação de uma Unidade de Medição Fasorial embarcada no processador: gerador de sinais, modelo de referência em dupla precisão e análise estatística

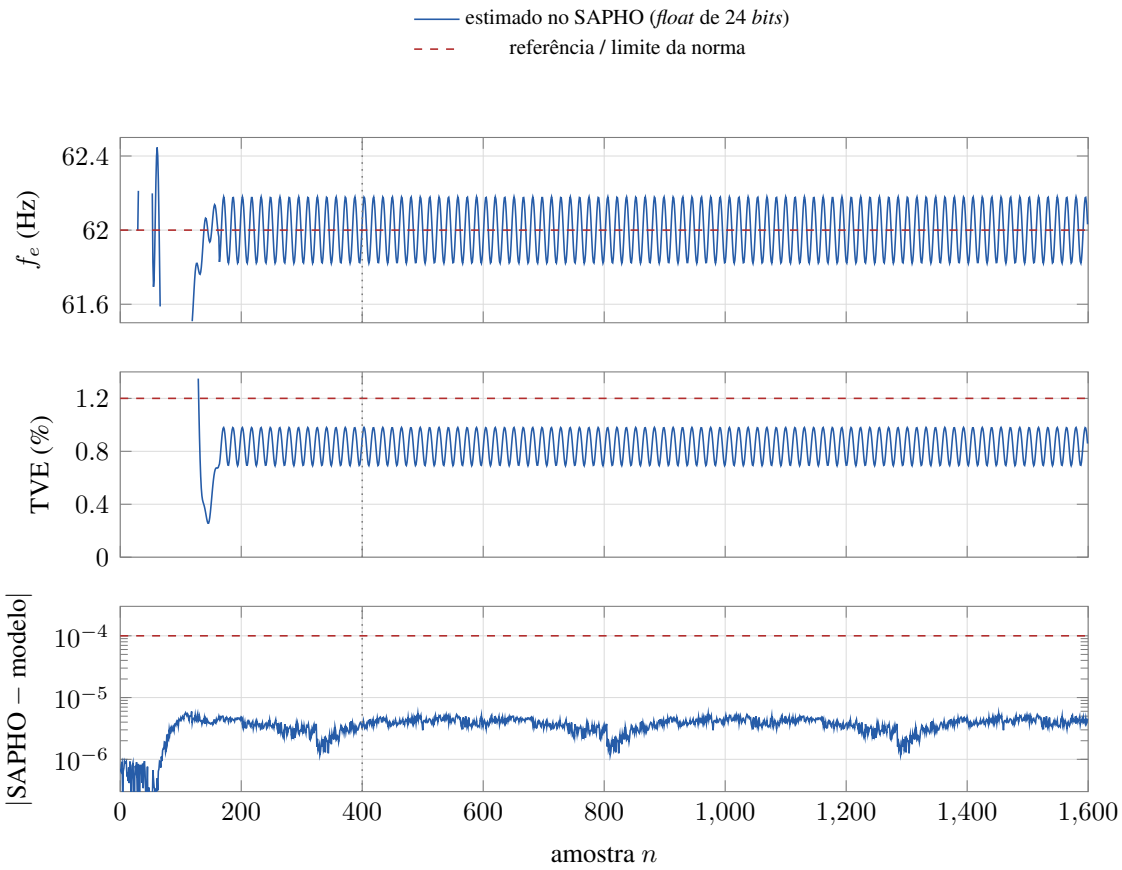


Figura 4: Verificação em dois níveis da PMU no SAPHO ao longo do ensaio. Acima, a frequência estimada; ao centro, o TVE contra a verdade analítica (nível 1); abaixo, o desvio absoluto em relação ao modelo de referência em dupla precisão (nível 2), em escala logarítmica. A linha tracejada marca a referência ou o limite aplicável e a linha pontilhada, o início da janela de regime ($n = 400$).
Fonte: Autores (2026)

conviveram em um único *testbench* Python, que confirmou a conformidade da implementação em regime permanente, nas métricas avaliadas da norma IEC/IEEE 60255-118-1, e a suficiência do *float* de 24 bits do SAPHO. Mais do que velocidade, o cocotb traz acessibilidade e produtividade à modelagem e à verificação de *hardware*, aproximando o fluxo do SAPHO de um público mais amplo de desenvolvedores. Essa acessibilidade estende-se à formação: o ecossistema SAPHO/AURORA já é adotado na cadeira de Dispositivos Lógicos Programáveis (DLP) da graduação em Engenharia Elétrica e na pós-graduação da UFJF, e a verificação em Python passa a integrar esse percurso, encurtando o caminho entre a teoria e o ensaio do circuito.

Agradecimentos

Os autores agradecem à Universidade Federal de Juiz de Fora (UFJF) e ao Núcleo de Instrumentação e Processamento de Sinais (NIPS-CERN) pela infraestrutura e pelo suporte contínuo, e ao CERN pela parceria científica que orienta as linhas de pesquisa do laboratório. Este trabalho contou com o apoio da Fundação de Amparo à Pesquisa do Estado de Minas Gerais (FAPEMIG), do Conselho Nacional de Desenvolvimento Científico e Tecnológico (CNPq) e da Coordenação de Aperfeiçoamento de Pessoal de Nível Superior (CAPES). O presente trabalho foi realizado com apoio da CAPES, Coordenação de Aperfeiçoamento de Pessoal de Nível Superior – Brasil, Código de Financiamento 001.

Declaração de uso de ferramentas de Inteligência Artificial generativa

Em conformidade com as diretrizes do evento, declara-se que o porte do algoritmo da PMU, do MATLAB para a linguagem C±, contou com o apoio da Aurora Intelligence, a assistente de inteligência artificial integrada à própria IDE AURORA e especializada no ecossistema SAPHO, conforme descrito na Seção 5. Todo o conteúdo científico, a interpretação dos resultados e as conclusões permanecem de responsabilidade dos autores.

Referências

Andrade Filho, L. M., Chrysthofer Arthur, A. A., Kapisch, E. B., de Castro, A. L. B., de Miranda Rimes, S., and Libotte, G. B. (2025). Implementação de fractal em FPGA usando o processador soft-core SAPHO. In *Anais do XXVIII Encontro Nacional de Modelagem Computacional (ENMC)*, Montes Claros, MG, Brasil.

cocotb contributors (2024). cocotb: A coroutine based cosimulation library for writing VHDL and Verilog testbenches in Python. `https://www.cocotb.org`. Acesso em: 23 jun. 2026.

Fonseca, N. d. L. (2025). Desenvolvimento de uma PMU: Estimação do fasor. Trabalho de conclusão de curso (engenharia elétrica), Universidade Federal de Juiz de Fora (UFJF), Juiz de Fora, MG, Brasil.

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., et al. (2020). Array programming with NumPy. *Nature*, 585:357–362.

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3):90–95.

IEC/IEEE (2018). IEC/IEEE international standard – measuring relays and protection equipment – part 118-1: Synchrophasor for power systems – measurements. IEC/IEEE 60255-118-1:2018.

Jiang, S., Pan, P., Ou, Y., and Batten, C. (2020). PyMTL3: A Python framework for open-source hardware modeling, generation, simulation, and verification. *IEEE Micro*, 40(4):58–66.

Kapisch, E. B., Aguiar, M. S., Andrade Filho, L. M., and Manso Silva, L. R. (2026). SAPHO—scalable-architecture processor for hardware optimization: An FPGA customizable implementation approach. *IEEE Access*, 14:6820–6832.

Phadke, A. G. and Thorp, J. S. (2008). *Synchronized Phasor Measurements and Their Applications*. Springer, New York, NY.

Ribeiro, L. d. G. and Andrade Filho, L. M. d. (2025). Simulador de pulsos do TileCal em tempo real utilizando o processador SAPHO e FPGA. In *Anais do XVII Simpósio Brasileiro de Automação Inteligente (SBAI)*.

Roscoe, A. J., Abdulhadi, I. F., and Burt, G. M. (2013). P and M class phasor measurement unit algorithms using adaptive cascaded filters. *IEEE Transactions on Power Delivery*, 28(3):1447–1459.

Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., et al. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17(3):261–272.

APPENDIX A: ENGLISH TITLE, ABSTRACT AND KEYWORDS

INTEGRATION OF COCOTB INTO THE AURORA IDE: COSIMULATION AND VERIFICATION OF HARDWARE MODELS IN PYTHON

Abstract. *Verifying an algorithm embedded in hardware is, in essence, a computational modelling problem: the same estimator exists as a synthesizable structural model, executed by a discrete-event simulator, and as a numerical reference model, and confidence in the circuit arises from the continuous confrontation between the two. This work presents the integration of cocotb (coroutine cosimulation testbench) into the AURORA IDE, part of the SAPHO soft-core processor ecosystem, which allows this confrontation to be written entirely in Python, in cosimulation with the Icarus Verilog and Verilator simulators, with the test routine recognized by the interface and run in a single click. The methodology is demonstrated on the verification of a Phasor Measurement Unit implemented on SAPHO according to the IEC/IEEE 60255-118-1 standard: stimulus generator, double-precision reference model, acquisition emulation and analysis coexist in a single testbench, which validates the hardware at two levels, against the standard's analytical phasor and against the numerical model. The first level confirmed a maximum Total Vector Error of 0.98%, within the normative limit; the second shows that the processor's 24-bit float reproduces the reference within* 5×10^{-6} *in the phasor, attributing the remaining error to the estimation method rather than to the hardware arithmetic.*

Keywords: *Digital system modelling and verification, Cosimulation, cocotb, SAPHO soft-core processor, Phasor Measurement Unit.*